EC 402 Final Project

Pendulum Stability Using Linear Motion

Aamir Sheikh
Reuben Then
Pree Simphliphan
aasheikh@bu.edu, benmthen@bu.edu, prees26@bu.edu

Abstract

This project investigates the stabilization of an inverted pendulum using linear motion control. The objective is to design and implement a control system capable of maintaining the pendulum in its unstable upright equilibrium. The system is modeled using classical mechanics, and a controller is designed based on the linearized dynamics. Experimental and simulation results are presented to evaluate system performance and stability.

1 Context of the Problem

The inverted pendulum is a classical benchmark problem in control systems due to its nonlinear and inherently unstable dynamics. Stabilizing the pendulum in its upright equilibrium requires continuous feedback and precise actuation, making it an ideal system for studying modern control techniques.

In this project, the pendulum is stabilized indirectly by controlling the linear motion of a cart to which the pendulum is attached. Instead of applying torque directly at the pivot, the system exploits inertial effects induced by cart motion to regulate the pendulum angle. This problem is closely aligned with various control systems topics, including nonlinear system modeling, linearization, state-space representation, and feedback control design.

The project also integrates both theoretical and practical components by combining simulation-based validation with real-time hardware implementation.

2 Problem Statement

The objective of this project is to design and implement a control system capable of stabilizing an inverted pendulum using only linear motion of its base. Unlike traditional pendulum control problems where torque is applied directly, this system requires controlling the cart position to indirectly influence the pendulum angle. This introduces additional complexity due to coupling between translational and rotational dynamics. The specific goals of the project are:

- Develop a dynamic model of the cart–pendulum system
- Design a feedback controller (PID-based) for stabilization
- Validate controller performance through simulation

- Implement the controller on physical hardware
- Compare simulated and experimental results

3 Methods and Analysis

3.1 Nonlinear Dynamic Modeling

The cart–pendulum system is modeled using Lagrangian mechanics, which provides a systematic framework for deriving equations of motion for coupled nonlinear systems. The generalized coordinates are chosen as:

$$q = [x, \theta] \tag{1}$$

where x represents the horizontal displacement of the cart and θ represents the pendulum angle measured from the downward vertical.

The state vector is defined as:

$$\mathbf{x} = [x, \dot{x}, \theta, \dot{\theta}]^T \tag{2}$$

3.1.1 Kinetic Energy

The total kinetic energy consists of two components: translational kinetic energy of the cart and rotational/translational energy of the pendulum.

$$T = \frac{1}{2}M\dot{x}^2 + \frac{1}{2}I_{\text{total}}\dot{\theta}^2 + \frac{1}{2}m(\dot{x}^2 + 2\dot{x}\dot{\theta}l\cos\theta) \tag{3}$$

The coupling term arises because the pendulum's center of mass moves relative to the cart, introducing velocity components in both x and θ directions.

3.1.2 Potential Energy

The gravitational potential energy of the pendulum is:

$$V = -mgl\cos\theta \tag{4}$$

This expression assumes zero potential at the lowest position, consistent with standard pendulum modeling.

3.1.3 Moment of Inertia

From the composite pendulum derivation (rod + disk), the total inertia about the pivot is:

$$I_{\text{total}} = I_{\text{rod}} + I_{\text{disk}} \tag{5}$$

Using the parallel axis theorem:

$$I_{\text{rod}} = \frac{1}{12}m_1L^2 + m_1\left(\frac{L}{2}\right)^2 \tag{6}$$

$$I_{\text{disk}} = \frac{1}{2}m_2r^2 + m_2(L+\delta)^2 \tag{7}$$

This formulation captures the distributed mass and ensures accurate modeling of rotational dynamics.

3.1.4 Equations of Motion

Applying the Euler–Lagrange equation:

$$\frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{q}}\right) - \frac{\partial \mathcal{L}}{\partial q} = Q \tag{8}$$

where $\mathcal{L} = T - V$, yields the nonlinear equations:

$$(M+m)\ddot{x} + ml\ddot{\theta}\cos\theta - ml\dot{\theta}^2\sin\theta = u \tag{9}$$

$$I_{\text{total}}\ddot{\theta} + ml\ddot{x}\cos\theta + mgl\sin\theta = 0 \tag{10}$$

These equations highlight the strong nonlinear coupling between translational and rotational motion.

3.2 Linearization

To design a stabilizing controller, the system is linearized about the upright equilibrium. Since the pendulum angle θ is measured from the downward vertical, the upright position corresponds to $\theta = \pi$. To simplify the analysis, a shifted angular coordinate is introduced:

$$\tilde{\theta} = \theta - \pi \tag{11}$$

where $\tilde{\theta}$ represents the deviation from the upright equilibrium.

For small deviations near the upright position, the small-angle approximations become

$$\sin(\theta) = \sin(\pi + \tilde{\theta}) \approx -\tilde{\theta}, \qquad \cos(\theta) = \cos(\pi + \tilde{\theta}) \approx -1 \tag{12}$$

Substituting these approximations into the nonlinear equations yields the linearized system about the upright equilibrium:

$$(M+m)\ddot{x} - ml\ddot{\tilde{\theta}} = u \tag{13}$$

$$I_{\text{total}}\ddot{\tilde{\theta}} - ml\ddot{x} - mgl\tilde{\theta} = 0 \tag{14}$$

These equations are then used to construct the state-space model for controller design.

3.3 Proportional-Integral-Derivative controller

A PID controller is used to stabilize the pendulum near the upright position:

$$u(t) = -K_p\theta - K_d\dot{\theta} - K_i\int \theta\, dt \tag{15}$$

The proportional term corrects angular deviation, the derivative term provides damping, and the integral term compensates steady-state error. In practice, the derivative term plays a dominant role in stabilizing oscillations.

3.4 Swing-Up Control Algorithm

Since the upright equilibrium is unstable, the pendulum must first be swung up using an energy-based control strategy.

3.4.1 Energy-Based Control Law

The total energy of the pendulum is defined as:

$$E = \frac{1}{2} I_{\text{total}} \dot{\theta}^2 + mgl(1 - \cos\theta) \tag{16}$$

The target energy corresponding to the upright position is:

$$E_{\text{target}} = 2mgl \tag{17}$$

The control objective is to regulate the energy error:

$$E_{\text{error}} = E_{\text{target}} - E \tag{18}$$

3.4.2 Control Energy Input Design

The applied force is designed as:

$$u \propto E_{\text{error}} \cdot \text{sign}(\dot{\theta} \cos\theta) \tag{19}$$

This ensures energy is injected when needed and removed when excessive.

3.4.3 Practical Enhancements

The implemented algorithm includes several practical modifications:

- **Startup Condition:** A sinusoidal input is applied initially to overcome static inertia.
- **Energy scaling:** Control effort is normalized to prevent excessive force.
- **Cart centering:** A restoring term prevents drift:

$$u_{\text{center}} = -k_x x - k_v \dot{x} \tag{20}$$

- **Rail protection:** Boundary forces prevent hardware damage.

3.4.4 Switching Logic

The system transitions to PID stabilization control mode when:

$$|\theta| < \theta_{\text{threshold}}, \quad |\dot{\theta}| < \omega_{\text{threshold}} \tag{21}$$

This ensures the pendulum is sufficiently close to the upright equilibrium before switching control strategies.

3.5 Hardware Design

To achieve pendulum balance, there are two main components to be built: the actuator that provides input to our system and the platform that supports the pendulum and senses its angle. The platform centers on a rod holding the pendulum on one end and a magnet on the other. The rod is supported on the platform by two bearings embedded in two pillars extruding out of the platform's base. This ensures that the pendulum can rotate freely without disturbance from the environment, allowing for easier controller design. On the magnet side of the rod, we have a magnetic encoder fastened to the platform's back wall. The magnetic encoder uses the magnet's magnetic field to acquire the angle of the pendulum, allowing us to determine the angle of our pendulum.

The actuator consists of a carriage mounted on an aluminum extrusion that serves as a rail. The carriage has screw holes to mount the pendulum's platform. The carriage has 3 MR105 bearings with POM casing to slide along the aluminum extrusion. The bearings are mounted to the carriage using M5x25 screws and an M5 locknut. Furthermore, the carriage also has slots for securing both ends of a GT2 timing belt and tensioning the belt. This design of the carriage allows the carriage to slide along the aluminum extrusion virtually with no friction, decreasing possible disturbances from the environment.

This carriage will be driven by a NEMA17 motor, which turns a timing pulley and drives the GT2 Timing Belt secured to the carriage. This stepper motor is also driven by a TMC2209 Motor Driver. This combination of motor and motor driver is because of the NEMA17's step size and the motor driver's ability to do high-resolution movement (microstepping). This choice allows us to be able to control the input to the motor very smoothly, making the input as continuous as possible while still being controlled discretely. For our microcontroller, we use a Teensy microcontroller that will receive input from our encoder, do calculations, and control input to the motor. We use this microcontroller because it can receive input, compute calculations, and deliver output much quicker than other microcontrollers for a relatively affordable price. This allows us to be able to control our output at faster intervals, allowing us to make our control as close to continuous as possible.

4 Simulations

Numerical simulations were conducted to validate the cart–pendulum dynamic model and evaluate the performance of the proposed swing-up and stabilization controllers. All simulations were implemented in Python using `NumPy` for numerical computation and `Matplotlib` for visualization and animation.

4.1 Simulation Models

- **Linearized model:** This model describes the pendulum dynamics near the upright equilibrium using small-angle approximation. It is used mainly for PID stabilization analysis and controller tuning.
- **Nonlinear model:** This model retains the full trigonometric dynamics of the pendulum and is used to evaluate behavior during large-angle motion and to compare against the linear approximation.

In addition, a dedicated swing-up simulation is included to evaluate the energy-based controller from the hanging-down initial position. The nonlinear and linearized PID simulations are implemented in `main_simulation_nonlinear.py` and `main_simulation_linearized.py`, while the swing-up simulation is implemented in `main_simulation.py`.

4.2 Numerical Integration

The state vector is defined as

$$\mathbf{x} = [x,\ \dot{x},\ \theta,\ \dot{\theta}]^T \tag{22}$$

where x is the cart position and θ is the pendulum angle. The system is simulated in state-space form as

$$\dot{\mathbf{x}} = f(\mathbf{x}, u) \tag{23}$$

where u is the commanded cart acceleration.

For the linearized and nonlinear PID models, the state equations are integrated using a fourth-order Runge–Kutta (RK4) method with a fixed time step of $\Delta t = 0.01$ s. For the swing-up-only simulation, the state update is implemented using step-by-step numerical integration with the same sampling interval. Rail limits and velocity reversal at impact are also included to preserve physically realistic cart motion.

4.3 Simulation Scenarios

- **Swing-up from the downward position:** The pendulum starts near $\theta = 0^\circ$ and an energy-based controller injects energy into the system.
- **PID stabilization near the upright equilibrium:** The pendulum starts close to the upright position with a small initial angular perturbation, and PID control is applied to regulate the pendulum back toward equilibrium.
- **Response to disturbance:** A disturbance in angle and angular velocity is applied during PID regulation to test the recovery capability of the controller.
- **Gain sensitivity analysis:** Separate gain sweeps are performed for K_p, K_i, and K_d in both the linearized and nonlinear models, and for the swing-up gains in the swing controller.

4.4 Performance Metrics

- Pendulum angle response and error from the upright position
- Cart position, cart velocity, pendulum angular velocity and angular acceleration
- First-pass overshoot and peak angle error during PID tuning
- Settling time
- Capture time for the swing-up controller, defined as the time required to bring the pendulum into the PID handoff region

4.5 Results and Discussion

The simulation results confirm that the control strategy is effective for both swing-up and stabilization. Including rail constraints and practical tuning analysis also makes the simulations more representative of a realistic implementation. During the swing-up stage, the energy-based controller drives the pendulum from the downward position toward the upright region while keeping

the cart motion within the rail limits. This behavior is shown in Appendix Fig. 2 and Fig. 3. The plots show energy injection into the pendulum motion.

For stabilization near the upright equilibrium, both the linearized and nonlinear PID simulations reduce the pendulum angle error after the initial transient. The time-response plots in Appendix Fig. 4 and Fig. 6 show that the pendulum angle remains close to the upright position after regulation begins, while the cart position and velocity remain bounded. The cart acceleration also remains limited, which means it doesn't require unrealistic control effort.

Comparing the two PID simulation models, the linearized response provides a cleaner view of the regulation behavior near equilibrium, while the nonlinear response captures effects that become important outside the small-angle regime. The nonlinear model gives a more realistic representation of transient motion and how the controller behaves under larger disturbance recovery.

The PID tuning plots in Appendix Fig. 8–10 and Fig. 11–13 show that controller performance depends strongly on the selected gains. The gain sweeps demonstrate a trade-off between response speed, overshoot, and peak error. In general, larger proportional gain tends to produce faster correction, while derivative gain improves damping and reduces oscillatory behavior. Integral gain helps reduce residual offset, but excessive integral action can produce a slower or less smooth response. These trends are visible in both the linearized and nonlinear tuning results, although the nonlinear plots show more variation because they retain the full pendulum dynamics.

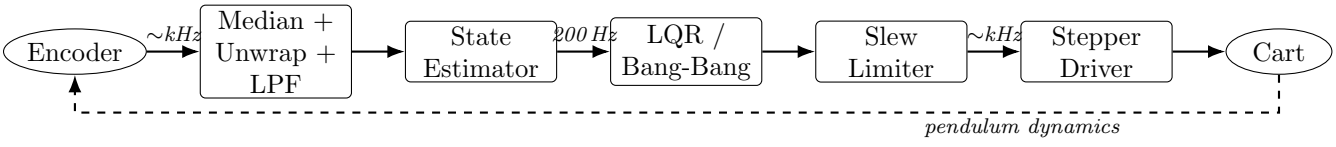
The swing-up tuning results in Appendix Fig. 14 and Fig. 15 show the trade-off between faster energy buildup and increased cart motion. Increasing the swing gain generally improves the ability of the controller to bring the pendulum toward the upright region, but it may also increase cart displacement. However, the startup gain doesn't affect the result of controller.

5 Embedded Implementation

Even though our simulation uses a PID controller, we have found that, on physical hardware, an LQR controller is easier to implement compared to the PID controller. Due to real-world variability, tuning PID needs comprehensive iterative testing. Thus, for the physical hardware, we have decided to proceed using an LQR controller. Even so, translating an LQR controller from a clean linearized model into firmware that actually balances a real pendulum exposes problems invisible to simulation: sensor noise that the controller amplifies, actuator dynamics that fight the control law, mechanical resonances excited by command discontinuities, and mode-transition logic that has to be self-healing in the face of imperfect catches. This section describes the embedded implementation choices that turned the theoretical controller into a working system, organized around the issues that arise when bridging the theory–hardware gap.

5.1 System Architecture

The embedded controller is structured as a set of concurrent soft real-time tasks running at different rates on a single microcontroller. Decoupling the rates is essential: the control law has different timing requirements than the sensor pipeline, which in turn has different requirements than the actuator. Forcing all three to run at the same rate compromises one or more of them.



The encoder is polled at the loop rate (kHz-class) because the angle-unwrap algorithm requires consecutive samples to be within half a revolution; a rapidly swinging pendulum can violate this if polling is too slow. The control law runs on a precise 200 Hz tick driven by a hardware timer, fast enough that the discrete-time derivative is a good approximation of the true angular velocity but slow enough to leave compute headroom. The stepper driver receives step pulses at whatever rate the slew-limited velocity command requires, also serviced at the loop rate. A 50 Hz logging task prints state to the serial monitor, throttled because uncontrolled printing will starve the control loop and corrupt timing.

5.2 Sensor Pipeline

A controller is a noise amplifier in the wrong frequency bands. Before tuning any gains, the sensor signal has to be conditioned so that the estimated state vector $[x, \dot{x}, \theta, \dot{\theta}]$ is dominated by signal, not noise. Several stages were necessary:

- **Hardware health checks first.** The magnetic encoder's status registers (magnet detect, AGC) were polled at startup before anything else. This caught a pre-software issue (wrong magnet type) that would have been mistaken for a control-tuning problem.
- **Median filter on raw counts.** A three-sample median rejects isolated I^2C glitches and magnetic spikes before any derivative is computed.
- **Continuous unwrap.** The encoder reports a 12-bit angle that wraps; consecutive samples are compared and shortest-path deltas are accumulated to produce a continuous angle that survives full pendulum revolutions during swing-up.
- **Mode-dependent low-pass filter.** The differentiated velocity $\dot{\theta}$ feeds a first-order IIR filter whose cutoff depends on the control mode: aggressive smoothing during stabilization (where small noise produces large commanded outputs near equilibrium) and lighter smoothing during swing-up (where peak detection requires faster response).

5.3 Actuator Modeling and Slew-Rate Limiting

A stepper motor in step/dir mode is fundamentally a position device, but the LQR control law produces a force-equivalent command. The bridge is to treat the stepper as an idealized velocity actuator: the control output is interpreted as a desired cart velocity, mapped to steps-per-second, and commanded directly. This works because the cart's velocity response is fast relative to the pendulum's natural frequency.

The naive form of this approach commands instantaneous velocity changes from the LQR output, which excites mechanical resonances in the belt, mounts, and pendulum hardware. The fix is a software slew-rate limiter that caps the commanded acceleration at a value tuned to the mechanical bandwidth of the system. This both quiets the system audibly and acts as a synthetic acceleration limit that the controller can rely on. Without it, swing-up to stabilization handoffs produce velocity slams that destabilize the catch.

A related tradeoff is the stepper driver's chopper mode. Quiet-but-low-torque modes are pleasant for stabilization, where motion is small, but limit swing-up authority. Forcing the high-torque mode in standalone configuration (a single-pin jumper) cost some acoustic comfort but unlocked the cart acceleration the pumping algorithm needed.

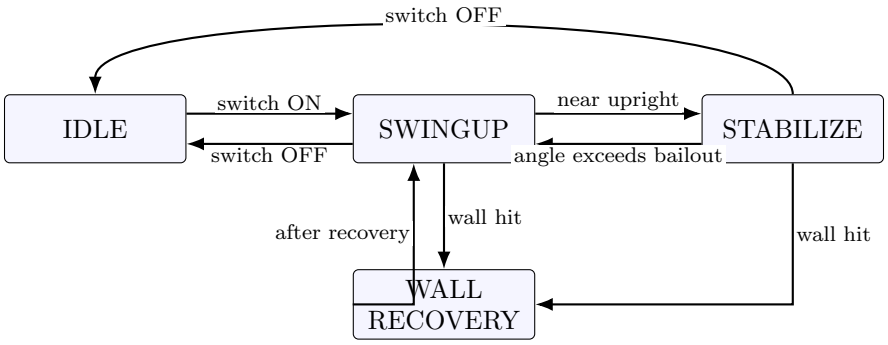
5.4 Calibration: Direct Measurement Beats Derivation

The most consequential implementation error in the project is a wrong assumption about the driver microstepping mode. Spec sheets and online documentation suggest one default; the actual hardware ran at twice that microstepping, meaning every constant computed from `steps_per_revolution` is off by a factor of two. The symptom is that soft cart-position limits were ignored entirely; the cause is arithmetic that nobody could have written incorrectly if calibrated against a physical ruler measurement.

The takeaway generalizes: the m-per-step (or whatever the analogous mechanical conversion factor is) should be determined by commanding a known number of steps and physically measuring the resulting motion. Calibrate, then derive. Spec-sheet-derived constants are first guesses, not ground truth.

5.5 Mode Logic: Self-Healing Behavior

The complete controller is a four-state machine. The transitions are designed so that the system retries indefinitely under user authority and only stops when the user disarms.



A failed catch returns to swing-up rather than to idle: the pendulum still has kinetic energy and the energy-pumping controller can pick up immediately. Hard wall hits trigger a brief reverse-velocity recovery state, not a disarm. The user, via the rocker switch, is the only authority that can halt the controller. This philosophy makes the system robust to imperfect catches and to the mechanical irregularities that a real prototype always exhibits.

5.6 Stabilization Augmentations

The textbook LQR control law $u = -Kx$ produces three pathologies on real hardware that each required a small but essential addition:

- **Deadband at equilibrium.** Sensor jitter at rest produces phantom $\dot{\theta}$ that the controller's velocity gain amplifies into saturating cart commands. A deadband zone, sized just above the noise floor, commands zero cart velocity when the state is essentially at equilibrium. Outside the deadband, full LQR engages.
- **Position-dependent soft wall.** LQR's cart-position gain is intentionally small to prioritize pendulum control. This means the cart drifts during balancing. A second position gain ramps in linearly as the cart approaches its travel limits, biasing the controller toward centering without compromising pendulum response near center.

- **Energy-aware swing-up.** Pure energy-error pumping can over-shoot, leaving the pendulum spinning past upright with too much kinetic energy to catch. Modulating the pumping direction by the sign of the energy error (pump when below target, brake when above) regulates total energy to the upright value, so the pendulum arrives at the catch zone with low velocity.

5.7 Generalizable Lessons

The transferable conclusions from this project apply broadly to any microcontroller-based control system bringing a textbook controller to real hardware:

- **Calibrate before deriving.** Mechanical conversion factors from spec sheets are first guesses; direct measurement against a physical reference catches assumption errors that code review cannot.
- **Match the controller paradigm to the regime.** Bang-bang control suits nonlinear, energy-pumping phases where timing matters more than magnitude; continuous-gain controllers like LQR suit linearized regimes near equilibrium. Combining purpose-built controllers via a state machine outperforms any single controller stretched across both.
- **Make the state machine self-healing.** Failed catches should retry, not give up; wall hits should recover, not disarm. The user should remain the only authority that halts a running controller.
- **Log everything; throttle the print.** A high-frequency internal log of state, mode, and commanded outputs is the single most valuable debugging tool, but the serial output rate must be capped so it cannot starve the control loop it is observing.

6 Tasks and Deliverables

Task	Task Name	Completion (%)	Assignee
1	Dynamic Modeling	100%	Aamir, Reuben
2	Simulation (Python)	100%	Pree
3	Controller Design & Tuning	100%	Pree
4	Hardware Integration	100%	Aamir, Reuben
5	Embedded Implementation (Teensy)	100%	Aamir, Reuben
6	Web Interface (Stretch Goal)	100%	Pree
7	Advanced Control (Stretch Goal)	100%	All

Table 1: Task completion status for the project.

Deliverables:

- Dynamic model of the system
- Simulation codebase and results demonstrating stabilization
- Physical prototype with controller implementation
- (Stretch Goal) Web Interface Prototype for visualization of different advanced controllers

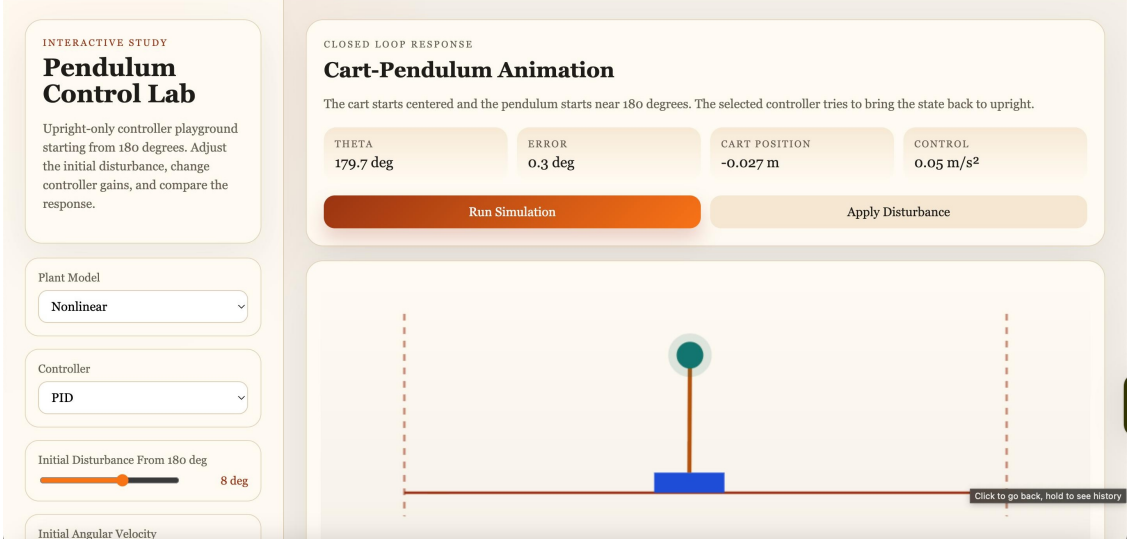


Figure 1: Prototype of Web Interface.

7 Conclusions

This project demonstrates the possibility of stabilizing an inverted pendulum using linear motion control. Through modeling, simulation, and hardware implementation, the system highlights the challenges of controlling nonlinear and unstable dynamics. Key lessons include the importance of accurate modeling, controller tuning, and handling real-world constraint such as noise, friction or tolerance of hardware design.

In addition, this project could be extended into an interactive educational platform for students studying control systems. By integrating the existing hardware and simulation framework with a web-based interface, users could modify system parameters, such as controller gains, system masses, friction coefficients, and visualize the resulting system response. The platform would allow students to develop understanding for control concepts by observing how different controllers affect stability, transient response, and robustness.

This extension would transform the project from a single application into a reusable teaching tool, bridging theoretical concepts from coursework with hands-on experimentation and visualization.

8 References

- `https://ctms.engin.umich.edu/CTMS/index.php?example=InvertedPendulum&section=SystemModeling`
- `https://www.mathworks.com/help/control/ug/control-of-an-inverted-pendulum-on-a-cart.html`
- `https://matplotlib.org/stable/api/animation_api.html`
- Codebase + Simulation and Web Video Demo: `https://drive.google.com/drive/folders/1XCX_93dNUsXKEkCYI4du-w3-SbZQDKYO?usp=drive_link`
- CAD Files: `https://drive.google.com/drive/folders/1e_iEIpG0Q2_2noUCbVb0a55uIDcrCcD7?usp=sharing`

- Generative AI usage: While all the advanced controllers, tuning and simulation has been researched and implemented by authors, the user interface of web interface has been prompted by Codex because authors are not specialized in UI/UX design and frontend development. However, it has been prompted and carefully debugged multiple files for the final prototype version.

9 Appendix: Graph Results from Simulation

This appendix collects all graph outputs generated from the simulation and tuning analysis. The graph files are stored under the path `saved_graphs/...`.

9.1 Swing-Phase Simulation

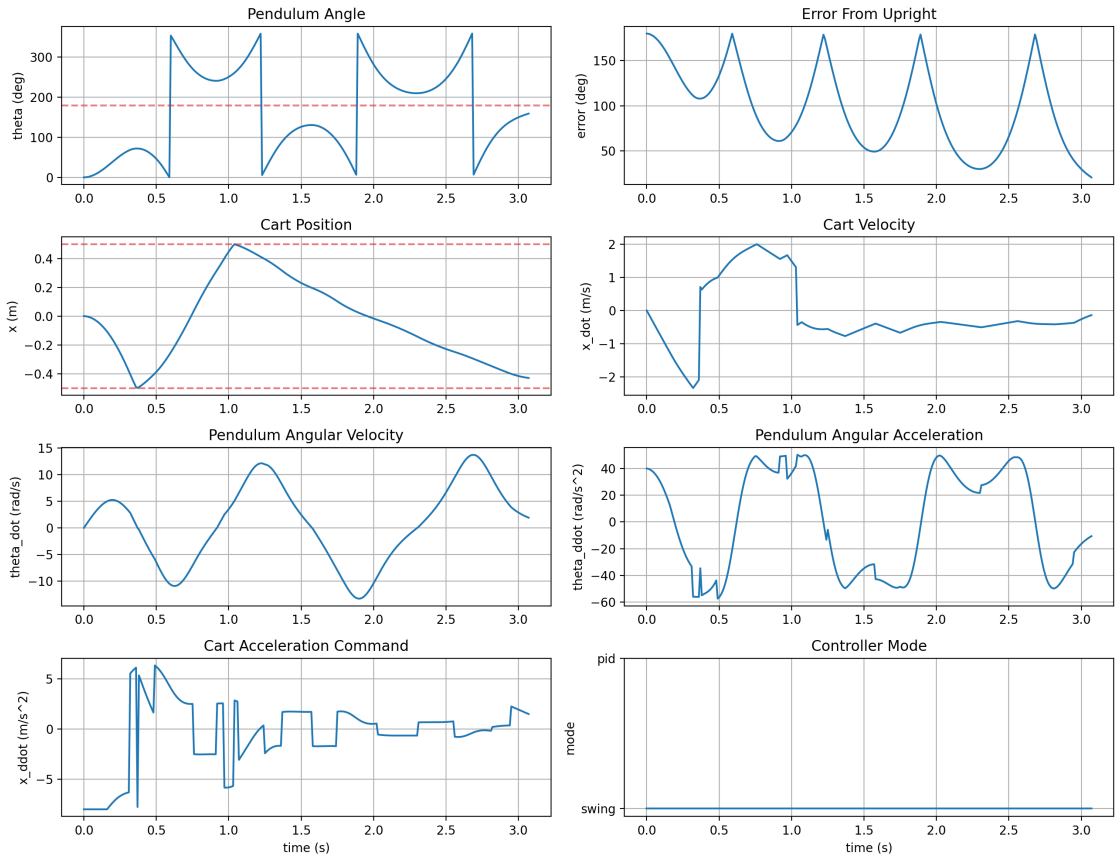


Figure 2: Summary plots for the swing-phase simulation.

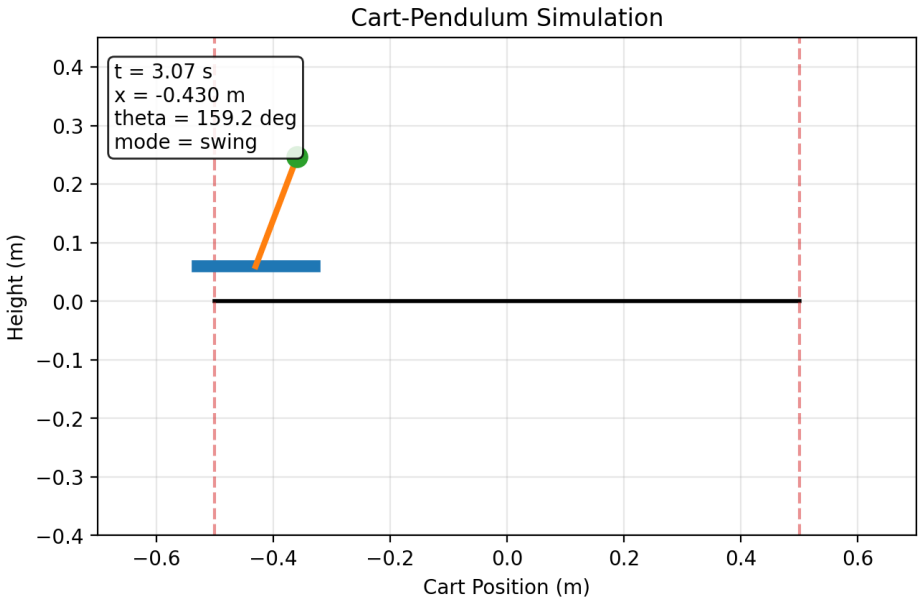


Figure 3: Final animation frame for the swing-phase simulation.

9.2 Linearized PID Simulation

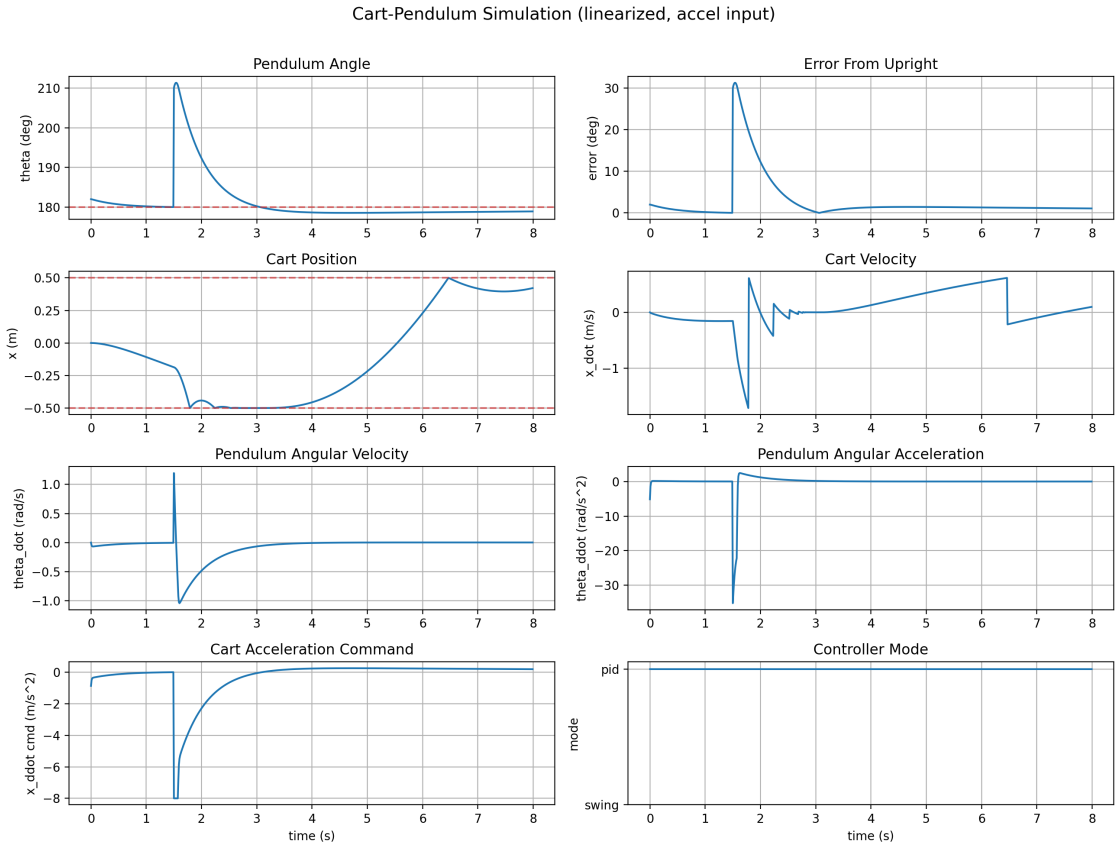


Figure 4: Summary plots for the linearized PID simulation.

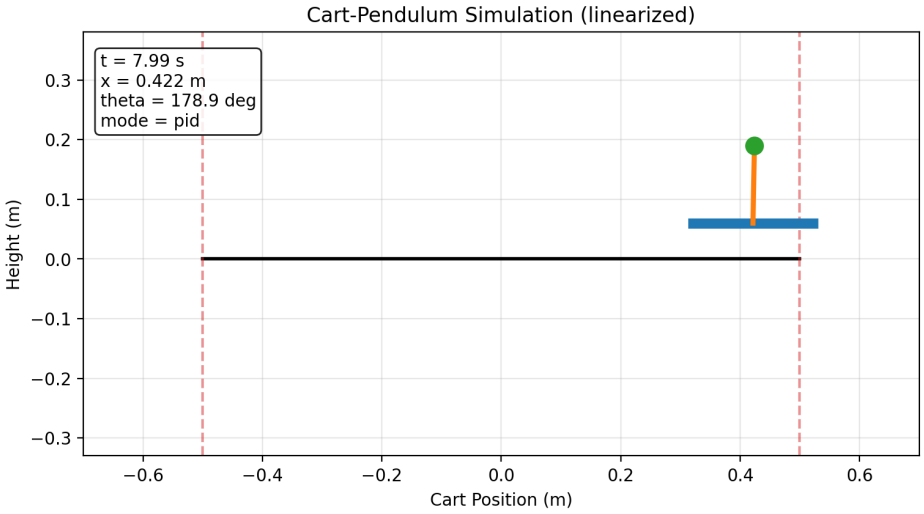


Figure 5: Final animation frame for the linearized PID simulation.

9.3 Nonlinear PID Simulation

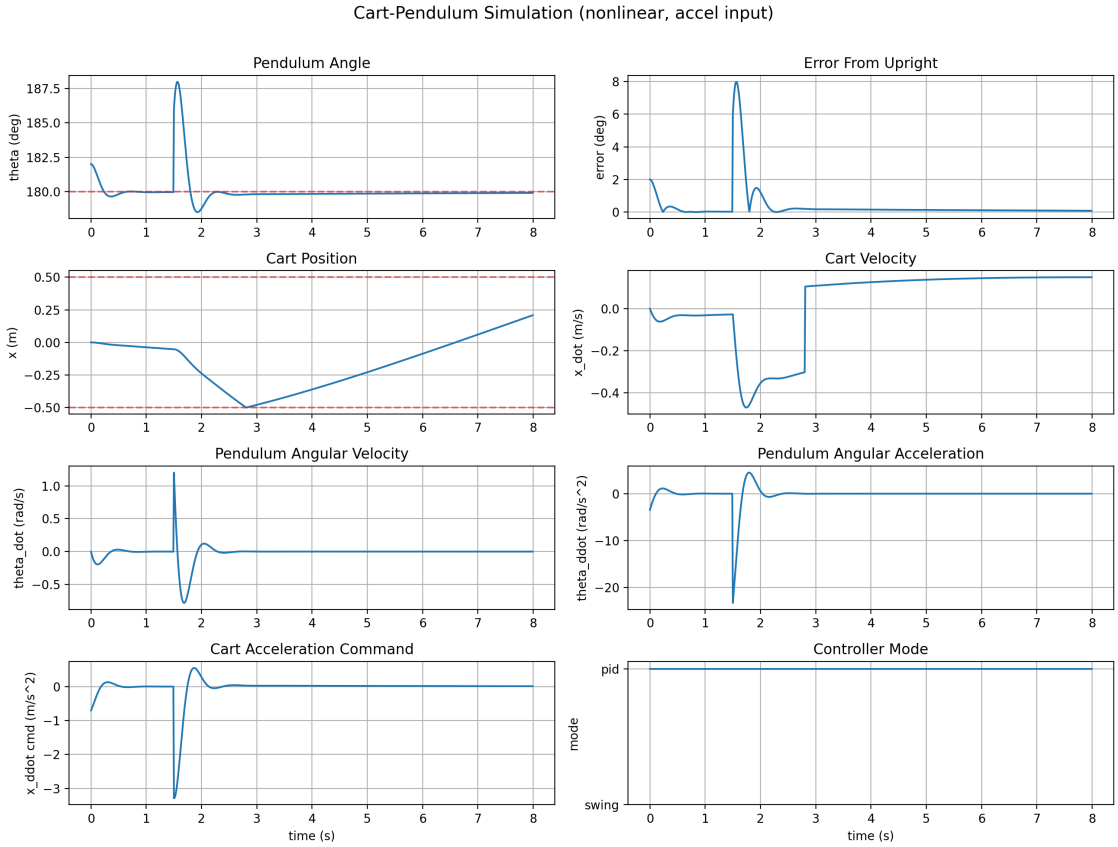


Figure 6: Summary plots for the nonlinear PID simulation.

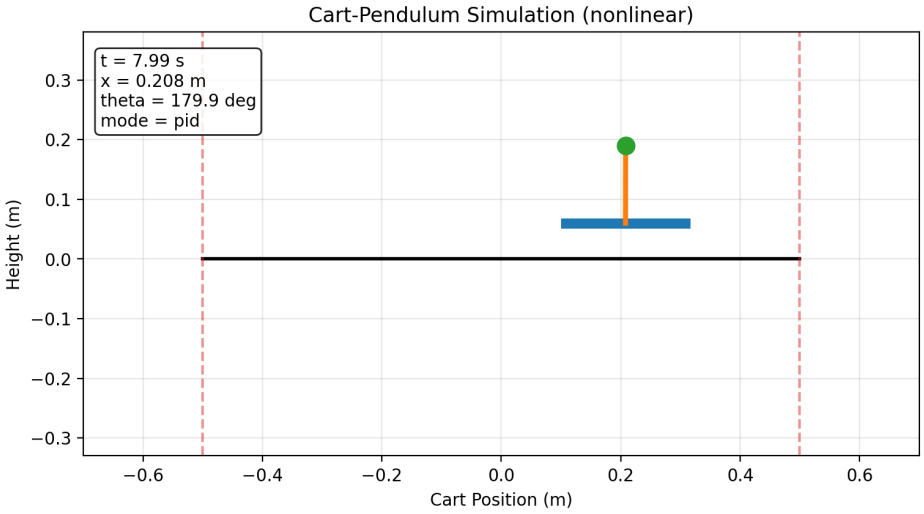


Figure 7: Final animation frame for the nonlinear PID simulation.

9.4 Linearized PID Tuning Analysis

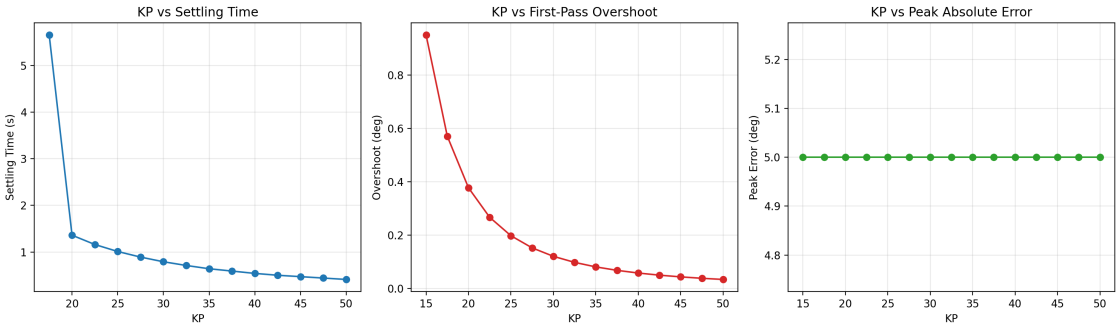


Figure 8: Linearized PID tuning results for proportional gain sweep.

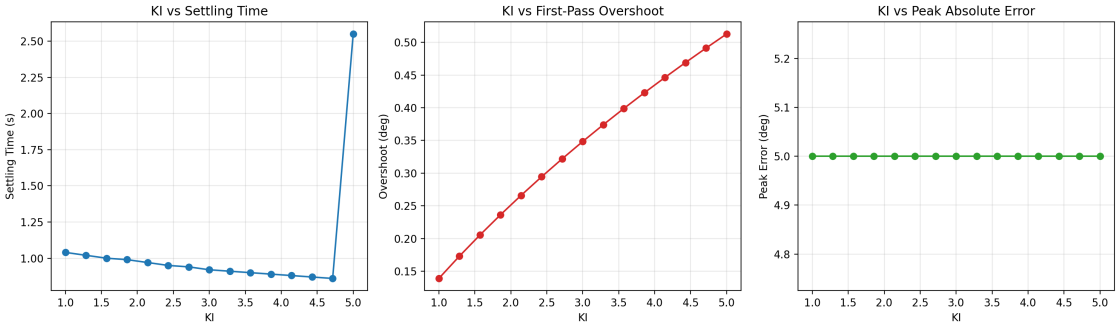


Figure 9: Linearized PID tuning results for integral gain sweep.

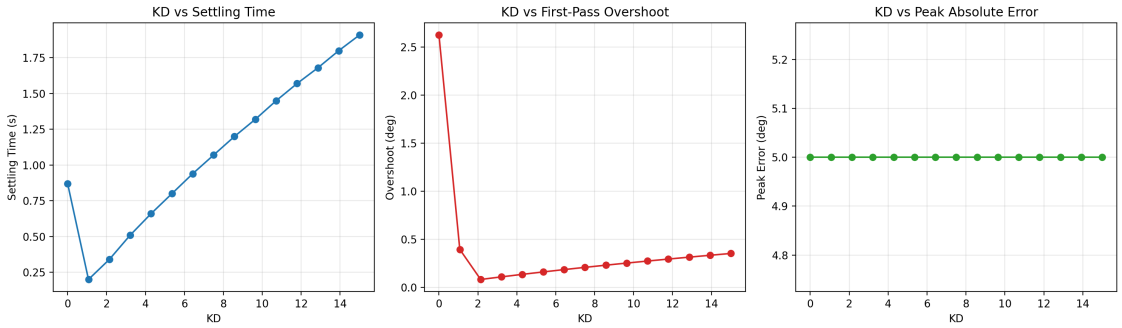


Figure 10: Linearized PID tuning results for derivative gain sweep.

9.5 Nonlinear PID Tuning Analysis

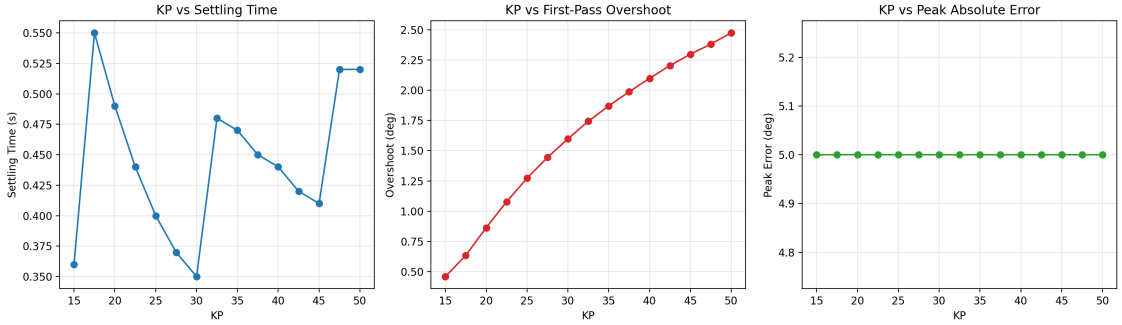


Figure 11: Nonlinear PID tuning results for proportional gain sweep.

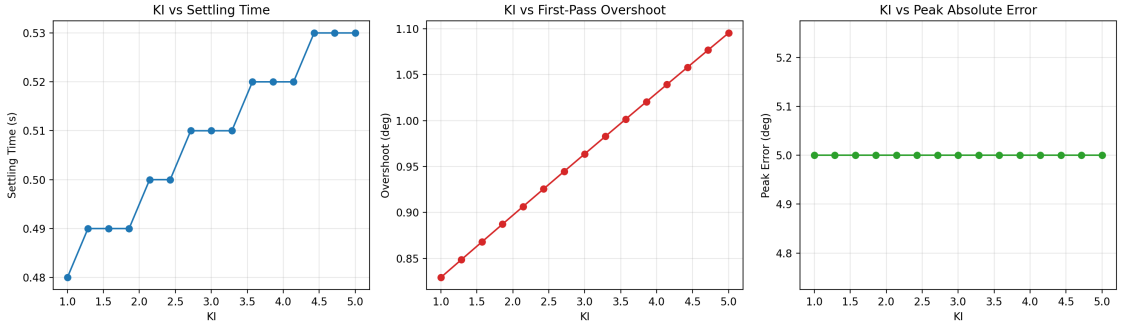


Figure 12: Nonlinear PID tuning results for integral gain sweep.

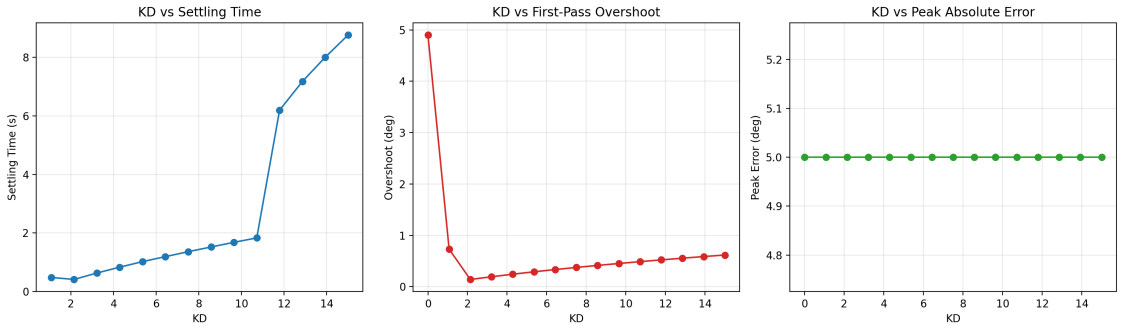


Figure 13: Nonlinear PID tuning results for derivative gain sweep.

9.6 Swing-Up Tuning Analysis

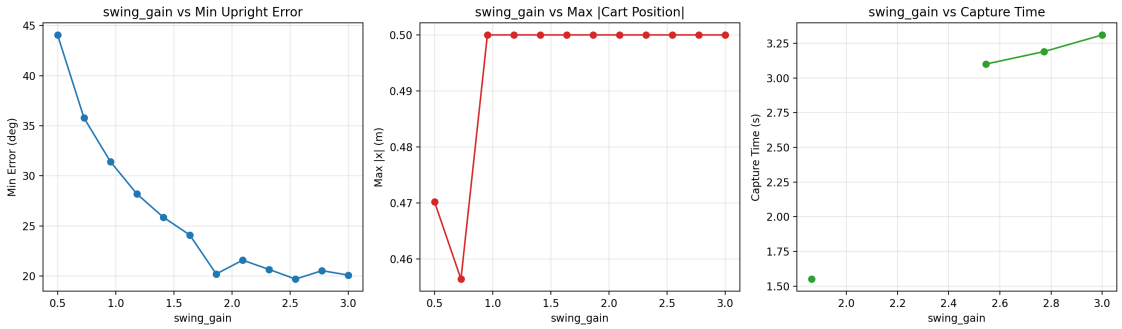


Figure 14: Swing-up tuning results for swing gain sweep.

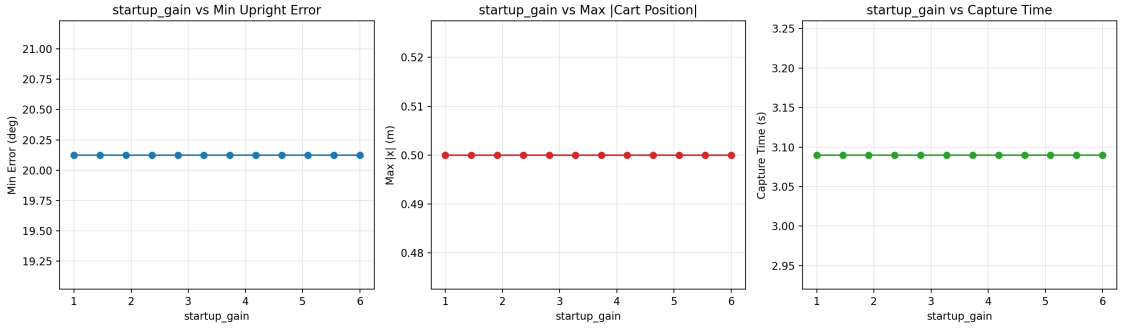


Figure 15: Swing-up tuning results for startup gain sweep.